

AP 2026 - Assignment 3

Lukas Vammen - TQR452
Jimmy Huynh - PLG852
Karan Inder Singh - GTC626

September 27, 2026

Contents

Introduction: 3

Questions and Answers: 4

 Question 1 4

 Question 2 4

 Question 3 4

 Question 4 5

 Question 5 5

Introduction:

One of the problems that required some thinking was how expressions should be grouped, especially when multiple operators are used together. This was solved by giving the operators different priorities; for example, multiplication is handled before addition, and in the correct direction. Keywords are also separated from normal variable names, which ensures words like "if", "let", and "try" are always treated as language-specific functions. The parser also supports function calls by reading expressions next to each other and combining them in the correct order.

The implementation is functionally correct and passes the test cases for the different parts of the language. These tests cover things such as numbers, variables, function applications, conditions, loops, let expressions, and try/catch statements.

Overall, the quality of the solution is good, as it covers the required functionality from the assignment and behaves as expected on the tested examples.

Questions and Answers:

1. Show the final and complete grammar with left-recursion removed and all ambiguities resolved.

We understood left-recursion as removing it completely, even if it does not follow the code anymore.

```

<Atom> ::= var
| int
| bool
| ( <Exp> )

<FExp> ::= <Atom>
| <Atom> <FExp>

<LExp> ::= <FExp>
| if <Exp> then <Exp> else <Exp>
| \ var -> <Exp>
| try <Exp> catch <Exp>
| let var = <Exp> in <Exp>
| loop var = <Exp> for var < <Exp> do <Exp>

<Exp4> ::= <LExp>
| print string <Atom>
| get <Atom>
| put <Atom> <Atom>

<Exp3> ::= <Exp4>
| <Exp4> ** <Exp3>

<Exp2> ::= <Exp3>
| <Exp3> * <Exp2>
| <Exp3> / <Exp2>

<Exp1> ::= <Exp2>
| <Exp2> + <Exp1>
| <Exp2> - <Exp1>

<Exp0> ::= <Exp1>
| <Exp1> == <Exp0>

<Exp> ::= <Exp0>

```

2. Why might or might not be problematic for your implementation that the grammar has operators * and ** where one is a prefix of the other?

It is not problematic since the implementation uses precedence. Exponentiation has a higher precedence level than multiplication. Therefore, the parser will always reach pExp2 in the code first, which contains the exponentiation.

3. Where in your parser is backtracking (try) strictly necessary, and where is it merely convenient? For one try you could remove, state what changes: the language recognised, the error messages, or the running time.

Try in "lKeyword" is strictly necessary:

Haskell

```

1 lKeyword :: String -> Parser ()
2 lKeyword s = lexeme \$ void \$ try \$ chunk s <*> notFollowedBy (satisfy isAlphaNum)

```

Because it consumes a keyword and then checks it using "notFollowedBy (satisfy isAlphaNum)". If this fails, it has to backtrack. This causes tests to error if they have a keyword as a prefix.

Try in "IVName" is strictly necessary:

Haskell

```
1 lVName :: Parser VName
2 lVName = lexeme \$ try \$ do
3   c <- satisfy isAlpha
4   cs <- many \$ satisfy isAlphaNum
5   let v = c : cs
6   if v `elem` keywords
7     then fail "Unexpected keyword"
8     else pure v
```

This causes all the if statement tests to error, since pExp parses the "then" with "IVName", but it is a keyword, so it must backtrack, which it can't without try.

4. Give an example of where in your parser it matters that choice is ordered (picks the first successful parse) (or equivalently for $<|>$, if you have used that instead of choice). If no such case occurs, explain why.

It does not matter, as operators have been separated into precedences, and try catches keyword/identifier ambiguity. No tests fail when changing them around.

5. Suppose APL is extended with comments notation (such as line comments or block comments as in Haskell). How would you implement this in your parser, and would it require many modifications?

It would not require many modifications. Lexeme already handles whitespace, so it can also handle comments. This can also be extended with block comments, if desired. So the only code changed is lexeme and parseAPL, with one or a few new functions to pass comments.